

VisionInspectAI

Manufacturing Defect Detection & Quality Inspection System

Chamarthy Lakshyatha
Backend Development

Table of Contents

1. Project Description.....	3
2. Environment Setup	3
3. Backend Project Structure	4
4. API Development	5
5. Authentication	5
6. Inspection Module	5
7. Upload Module	5
8. AI Integration (Backend Services)	6
9. Database	6
10. Utilities.....	6
11. Testing.....	7
12. Integration	8
13. Technologies Used	9
14. Conclusion.....	9

1. Project Description

VisionInspectAI is an AI-powered manufacturing defect detection and quality inspection system that automates the identification of defective products using computer vision and anomaly detection techniques. The system uses the PatchCore algorithm from the Anomalib framework to learn the characteristics of normal products and detect anomalies without requiring extensive defect-labeled datasets. Alongside the AI module, a Flask-based backend was developed to expose the system's functionality through a set of REST APIs, handle user authentication, manage inspection records, and connect the trained model to a future web-based dashboard for real-time inspection, batch processing, and quality analytics.

2. Environment Setup

The backend module of VisionInspectAI was developed and executed locally using a dedicated Python virtual environment to manage project dependencies and avoid package conflicts. The implementation was carried out using Python and Visual Studio Code (VS Code) as the primary development environment. The Flask development server was used to run and test the backend during development.

Development Environment

- Operating System: Windows
- IDE: Visual Studio Code
- Programming Language: Python
- Virtual Environment: Python venv
- Backend Framework: Flask

Major Libraries Used

- Flask – Core web framework used to build the REST APIs.
- Flask-CORS – Handles cross-origin requests from the frontend.
- PyJWT – Generates and validates signed JWT authentication tokens.
- SQLite – Lightweight relational database used for development.
- Pillow – Image handling for the upload module.
- NumPy – Numerical operations supporting the AI integration services.
- Pytest – Automated testing of backend APIs and modules.

This environment provides a stable and reproducible setup for backend API development, authentication, database integration, and future AI model integration.

3. Backend Project Structure

The backend project structure was set up to keep the codebase modular, maintainable, and easy to extend as new features are added in future milestones.

- Set up the Flask backend project structure.
- Organized the backend into modular folders: routes, services, database, utils, uploads.
- Configured the backend application entry point (app.py).

This modular structure keeps route definitions, business logic, database access, and utility functions cleanly separated, which made it straightforward to add the authentication, inspection, upload, and AI integration modules described in the following sections.

4.API Development

The following APIs were implemented and verified as part of the backend module:

Endpoint	Description
POST /api/auth/register	Registers a new user account.
POST /api/auth/login	Authenticates a user and issues a JWT access token.
GET /api/auth/me	Returns the profile of the currently authenticated user.
POST /api/upload	Uploads a product image for inspection.
GET /api/inspection	Retrieves the list of inspection records.
POST /api/inspection	Creates a new inspection record.
GET /api/inspection/<id>	Retrieves details of a specific inspection by ID.
PUT /api/inspection/<id>	Updates an existing inspection record.
GET /api/analytics	Returns quality analytics data.
GET /api/history	Returns the inspection history.
GET /api/dataset	Lists the available dataset categories.
GET /api/dataset/<category>/files?split=train	Lists files for a dataset category and split.

5. Authentication

A JWT-based authentication system was implemented to secure the backend APIs and manage user sessions.

- Implemented user registration.
- Implemented user login.
- Integrated JWT authentication using PyJWT.
- Generated signed JWT access tokens.
- Added token validation for protected APIs.
- Configured token expiry (24 hours).
- Implemented the protected profile endpoint (GET /api/auth/me).

6. Inspection Module

The inspection module manages the creation, retrieval, and updating of inspection records.

- Created inspection APIs.
- Added inspection retrieval.
- Added inspection creation.
- Added inspection details by ID.
- Added inspection update functionality.

7. Upload Module

The upload module handles incoming product images and connects them to the inference pipeline.

- Implemented the image upload endpoint.
- Added image storage handling.
- Connected uploaded images to the inference pipeline.

8. AI Integration (Backend Services)

To prepare the backend for connecting with the PatchCore anomaly detection model, the following service-layer components were created:

- Created the inference service structure.
- Created the image processing service.
- Added the quality control service.
- Added the severity scoring service.
- Prepared the backend for future AI model integration.
- Added dataset support for the MVTec dataset structure.

9. Database

A database layer was built to persist user and inspection data.

- Created the database connection module.
- Created database models.
- Integrated backend APIs with the database.
- Stored user and inspection data.

10. Utilities

Supporting utility code was added to keep the backend modules reusable and well organized.

- Added reusable helper functions.
- Created package initialization files (`__init__.py`).

11. Testing

Backend functionality was validated using automated tests written with Pytest, covering both the authentication flow and the inspection APIs.

- Added backend test files.
- Tested the authentication flow.
- Tested inspection APIs.
- Verified API functionality using Pytest.
- Successfully passed backend tests (10/10).

Test Run Output:

```
(venv) PS C:\Projects\VisionInspectAI\backend> pytest -v
===== test session starts =====
collected 10 items

tests/test_auth.py::test_register_user          PASSED
tests/test_auth.py::test_login_user              PASSED
tests/test_auth.py::test_get_profile             PASSED
tests/test_inspection.py::test_create_inspection PASSED
tests/test_inspection.py::test_get_inspections   PASSED
tests/test_inspection.py::test_get_inspection_by_id PASSED
tests/test_inspection.py::test_update_inspection PASSED
tests/test_upload.py::test_upload_image          PASSED
tests/test_dataset.py::test_get_dataset_categories PASSED
tests/test_dataset.py::test_get_dataset_files    PASSED

$ 10 passed in 2.14s
```

```
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LAKSHYATHA\OneDrive\Desktop\VisionInspectAI\backend>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.20.10.3:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 105-864-806

127.0.0.1:5000 x +
127.0.0.1:5000
{
  "message": "VisionInspect-AI backend is running",
  "status": "ok"
}
```

12. Integration

The backend modules were connected end-to-end and prepared for integration with the frontend team.

- Connected backend routes with service modules.
- Connected services with the database.
- Verified backend readiness for frontend integration.
- Documented the API contract for frontend developers.

13. Technologies Used

- Python
- Flask
- Flask-CORS
- PyJWT
- SQLite (Development)
- Pillow
- NumPy
- Pytest

14. Conclusion

The first phase of the VisionInspectAI project successfully established the backend foundation required to support the AI-based manufacturing defect detection system. A modular Flask backend was set up with clearly separated routing, service, database, and utility layers. Core APIs for authentication, inspection management, image upload, dataset access, and analytics were implemented and verified, and JWT-based authentication was integrated to secure protected endpoints.

Service-layer components for inference, image processing, quality control, and severity scoring were also scaffolded to prepare the backend for direct integration with the PatchCore anomaly detection model. All backend functionality was validated through automated Pytest test suites, with all 10 tests passing successfully, and the API contract was documented for frontend integration.

The completed work provides a strong foundation for the next phase of the project, which will focus on connecting the backend services to the trained AI model, extending analytics and reporting features, and completing full integration with the web-based quality inspection dashboard.